

Este material de estudio profundiza en los conceptos introductorios de ASP.NET Core MVC.

Material de Estudio: Introducción a ASP.NET Core MVC

Profesor: Jorge Izaguirre

Materia: Programación .NET

Curso: 6to Año - TÉCNICO EN PROGRAMACIÓN

Parte 1: Entendiendo MVC y los Fundamentos de ASP.NET Core

(1.1) De las Páginas Estáticas a las Aplicaciones Web Dinámicas

En el eje anterior, aprendimos a crear páginas web estáticas usando HTML para la estructura y CSS para el estilo. Estas páginas son como documentos: muestran siempre el mismo contenido a cualquier usuario que las visite. Sin embargo, la mayoría de las aplicaciones web que usamos hoy son **dinámicas**: su contenido cambia según quién las visita, qué acciones realiza el usuario, o qué información hay en una base de datos (ej: redes sociales, tiendas online, correo electrónico).

Para construir estas aplicaciones dinámicas y complejas de forma organizada y mantenible, necesitamos una **arquitectura** de software. Una de las más populares y efectivas para aplicaciones web es la arquitectura **Modelo-Vista-Controlador (MVC)**.

(1.2) El Patrón Arquitectónico Modelo-Vista-Controlador (MVC)

MVC no es un lenguaje ni un framework específico, sino un **patrón de diseño arquitectónico** que propone dividir una aplicación en tres componentes principales interconectados, cada uno con una responsabilidad bien definida.

- **¿Por Qué Usar MVC? El Poder de la Separación de Responsabilidades (Separation of Concerns - SoC):**
 - *¿Cuál es el problema sin MVC?* En enfoques más antiguos, era común mezclar la lógica para acceder a datos, la lógica de negocio (reglas), la generación de HTML y el manejo de la entrada del usuario, todo junto en un mismo archivo o de forma desorganizada. Esto genera "código espagueti", difícil de entender, modificar, probar y escalar.
 - *La Solución MVC:* Divide la aplicación en capas lógicas:
 1. **Modelo (Model):**
 - *Rol:* Representa los **datos** de la aplicación (ej: las clases *Alumno*, *Producto*, *Pedido*) y la **lógica de negocio** asociada a esos datos (reglas de validación, cálculos, operaciones complejas). Es el "cerebro" de la aplicación en cuanto a la información y sus reglas.
 - *Características:* **No sabe nada** sobre cómo se presentarán los datos al usuario (la Vista) ni cómo llegan las peticiones (el Controlador). Es independiente de la interfaz.
 - *Ejemplo Conceptual:* En una tienda online, el Modelo incluiría clases como *Producto* (con propiedades como *Nombre*, *Precio*, *Stock*) y métodos como *CalcularPrecioConIVA()*, *HayStockSuficiente()*.
 2. **Vista (View):**

- **Rol:** Es responsable de **presentar los datos** al usuario, generalmente generando la interfaz de usuario (HTML). Su principal trabajo es mostrar la información que recibe (normalmente del Modelo, a través del Controlador).
- **Características:** **No contiene lógica de negocio compleja.** Debe ser lo más "tonta" posible en cuanto a decisiones. Puede contener lógica de presentación simple (ej: bucles para mostrar una lista, condicionales para mostrar/ocultar algo). Utiliza los datos que le pasan para construir el HTML.
- **Ejemplo Conceptual:** Una página que muestra los detalles de un producto, tomando el *Nombre*, *Precio* y *Stock* que le llegan y colocándolos en etiquetas HTML apropiadas.

3. Controlador (Controller):

- **Rol:** Actúa como el **intermediario** o el "director de orquesta". Es el primer punto de entrada para manejar las **peticiones del usuario** (HTTP Requests).
- **Responsabilidades:**
 - Recibe la entrada del usuario (datos de formularios, parámetros en la URL).
 - Interactúa con el **Modelo** para realizar acciones (ej: pedirle al modelo que guarde un nuevo producto, que obtenga la lista de clientes, que valide un usuario).
 - Selecciona la **Vista** apropiada que debe mostrarse como respuesta.
 - (Opcional) Pasa los datos necesarios desde el Modelo hacia la Vista para que esta pueda mostrarlos.
- **Ejemplo Conceptual:** Cuando buscas productos en la tienda, el *ProductoController* recibe tu búsqueda, le pide al Modelo que busque los productos que coinciden, y luego elige la Vista "ResultadosDeBusqueda" pasándole la lista de productos encontrados.

◦ Beneficios de esta Separación:

- **Mantenibilidad:** Cambiar la interfaz de usuario (Vista) no debería requerir grandes cambios en la lógica de negocio (Modelo). Cambiar cómo se almacenan los datos (Modelo) no debería romper la Vista si el Controlador adapta el pasaje de datos.
- **Testabilidad:** Cada componente (especialmente Modelo y Controlador) se puede probar de forma más aislada.
- **Trabajo en Paralelo:** Diferentes equipos o personas pueden trabajar en diferentes componentes (diseñadores en Vistas, desarrolladores backend en Modelo/Controlador) con menor interferencia.
- **Organización:** El código queda naturalmente estructurado y es más fácil de entender.

• Flujo Típico de una Petición MVC:

1. El usuario realiza una acción en el navegador (ej: hace clic en un enlace, envía un formulario).
2. El navegador envía una **Petición HTTP** al servidor web.
3. El framework ASP.NET Core recibe la petición y, mediante el sistema de **Routing** (enrutamiento), determina qué **Controlador** y qué **Método de Acción** dentro de ese controlador debe manejarla.
4. Se crea (o reutiliza) una instancia del Controlador y se ejecuta el Método de Acción correspondiente.
5. La Acción interactúa con el **Modelo** (ej: lee datos de una base de datos, realiza un cálculo, valida información).

6. La Acción elige una **Vista** para generar la respuesta.
7. La Acción (generalmente) pasa los datos necesarios (obtenidos del Modelo) a la Vista seleccionada.
8. La **Vista** utiliza esos datos para generar dinámicamente el código **HTML**.
9. El HTML generado se envía de vuelta al navegador como parte de la **Respuesta HTTP**.
10. El navegador interpreta el HTML y muestra la página al usuario.

(1.3) ASP.NET Core: El Framework MVC de Microsoft

ASP.NET Core es el framework de Microsoft para construir aplicaciones web modernas, APIs y microservicios. Es la evolución de ASP.NET (que corría solo en Windows).

- *Características Clave:*
 - **Multiplataforma:** Corre en Windows, macOS y Linux.
 - **Open Source:** Su código fuente está disponible públicamente.
 - **Alto Rendimiento:** Diseñado para ser rápido y eficiente.
 - **Modular:** Construido con componentes (middleware) que se pueden añadir o quitar.
 - **Inyección de Dependencias (DI):** Un patrón incorporado para gestionar las dependencias entre clases, haciendo el código más desacoplado y testeable (lo veremos más adelante).
 - **Soporte para MVC y Razor Pages:** Ofrece MVC como patrón principal, pero también Razor Pages como alternativa para aplicaciones más centradas en páginas.
- **El Pipeline de Middleware:**
 - ¿Qué es? Imagina una línea de ensamblaje por la que pasa cada petición HTTP que llega a tu aplicación. Cada estación en la línea es un **middleware**.
 - ¿Qué hace un middleware? Es un componente de software que puede:
 - Examinar la petición HTTP entrante.
 - Modificar la petición antes de pasarla al siguiente middleware.
 - Generar una respuesta HTTP directamente (cortocircuitando el resto del pipeline).
 - Procesar la respuesta HTTP generada por middlewares posteriores.
 - ¿Cómo se configura? En el archivo **Program.cs** (en .NET 6 y posteriores), se usa el objeto **app** (de tipo **WebApplication**) para añadir middlewares en un orden específico. **¡El orden importa mucho!**
 - *Ejemplo Conceptual de Pipeline:*
 1. **app.UseExceptionHandler("/Error");** (Manejo de errores: si algo falla después, redirige a una página de error).
 2. **app.UseStaticFiles();** (Sirve archivos estáticos como CSS, JS, imágenes desde la carpeta **wwwroot**). Si pide un archivo estático, lo sirve y termina la petición aquí.
 3. **app.UseRouting();** (Determina qué endpoint -ej: acción MVC- debería manejar la petición basado en la URL, pero todavía no lo ejecuta).
 4. **app.UseAuthentication();** (Identifica quién es el usuario, si está logueado).
 5. **app.UseAuthorization();** (Verifica si el usuario identificado tiene permisos para acceder al recurso solicitado).
 6. **app.MapControllerRoute(...);** (Define cómo las URLs mapean a Controllers/Actions y efectivamente ejecuta el endpoint correspondiente si el routing tuvo éxito).
- **Configuración (Program.cs en .NET 6+):**

- Este archivo es el punto de entrada y configuración principal.
- **Fase 1: Configuración de Servicios (Inyección de Dependencias):**
 - Se usa `builder.Services` (un `IServiceCollection`). Aquí registras los "servicios" que tu aplicación necesitará. Un servicio es una clase que realiza una tarea específica (ej: acceso a base de datos, envío de emails, los propios servicios de MVC).
 - Ejemplo clave: `builder.Services.AddControllersWithViews();` registra todos los servicios necesarios para que funcione MVC (controladores, vistas, motor Razor, etc.).
- **Fase 2: Configuración del Pipeline de Middleware:**
 - Se usa `app` (un `WebApplication`). Aquí defines qué middlewares se usarán y en qué orden, como vimos en el ejemplo anterior.
 - Ejemplo clave (Routing MVC):

```
// Habilita el middleware que encuentra el endpoint (pero no lo ejecuta)
app.UseRouting();

// ... otros middlewares como autenticación/autorización ...

// Define cómo las URLs mapean a acciones de controlador y ejecuta el endpoint
app.MapControllerRoute(
    name: "default", // Un nombre para la ruta
    pattern: "{controller=Home}/{action=Index}/{id?}"); // La plantilla de ruta
```

- *Inyección de Dependencias (DI) - Concepto:* En lugar de que una clase (ej: un Controlador) cree directamente los objetos que necesita (ej: un servicio para acceder a la base de datos), la clase declara que *necesita* ese servicio (generalmente pidiéndolo en su constructor). El "contenedor de DI" de ASP.NET Core se encarga de crear (o reutilizar) una instancia del servicio solicitado y "inyectársela" automáticamente a la clase cuando esta se crea. *¿Por qué?* Desacopla las clases, facilita el reemplazo de implementaciones y mejora enormemente la testabilidad. Lo veremos más en detalle al conectar con bases de datos.

(1.4) Creando tu Primer Proyecto ASP.NET Core MVC

1. **Abre Visual Studio.**
2. **Ve a Archivo > Nuevo > Proyecto.**
3. **Busca y selecciona la plantilla:** "Aplicación web ASP.NET Core". Asegúrate de que en las etiquetas o descripción indique "(Modelo-Vista-Controlador)" o "MVC". **No elijas** "API web", "Aplicación de página única (SPA)" o "Aplicación web Blazor" por ahora.
4. **Configura el proyecto:** Dale un nombre (ej: `MiPrimeraAppMVC`), elige una ubicación, y selecciona la versión de .NET Framework que tengas instalada (ej: .NET 8.0, .NET 7.0, .NET 6.0 LTS - Long Term Support).
5. **Tipo de autenticación:** Por ahora, déjalo en "Ninguno".
6. **Configurar para HTTPS:** Déjalo marcado (es la práctica estándar).
7. **Haz clic en "Crear".** Visual Studio generará un proyecto con una estructura básica y algo de código de ejemplo.

- **Explorando la Estructura:**

- **Controllers/**: Aquí vivirán tus clases Controlador (ej: **HomeController.cs** viene por defecto).
- **Models/**: Aquí pondrás las clases que representan tus datos y lógica de negocio (ej: **Alumno.cs**, **Producto.cs**). Inicialmente puede estar vacía o tener una clase simple como **ErrorMessage.cs**.
- **Views/**: Contiene los archivos **.cshtml** (Vistas Razor). Está organizada en subcarpetas:
 - **Views/Home/**: Vistas específicas para **HomeController** (ej: **Index.cshtml**, **Privacy.cshtml**).
 - **Views/Shared/**: Vistas que pueden ser usadas por múltiples controladores (ej: **_Layout.cshtml**, **_ValidationScriptsPartial.cshtml**, **Error.cshtml**). El guion bajo **_** al inicio es una convención para indicar que no son vistas directamente accesibles por una URL, sino parciales o layouts.
- **wwwroot/**: **La única carpeta cuyo contenido es accesible directamente por el navegador.** Aquí van tus archivos estáticos:
 - **css/**: Hojas de estilo CSS.
 - **js/**: Archivos JavaScript.
 - **lib/**: Librerías de front-end (ej: Bootstrap, jQuery, si las usaras).
 - Imágenes, fuentes, etc.
- **Program.cs** (o **Startup.cs** + **Program.cs** en .NET < 6): El archivo principal de configuración y arranque.
- **appsettings.json** (y **appsettings.Development.json**): Archivos para guardar configuraciones de la aplicación (cadenas de conexión a BD, claves de API, etc.).

- **Ejecutando la Aplicación:** Presiona **F5** o el botón "Iniciar" (con forma de triángulo verde). Visual Studio compilará el proyecto, iniciará un servidor web de desarrollo (Kestrel) y abrirá tu navegador por defecto apuntando a la URL de la aplicación (usualmente algo como **https://localhost:7xxx**). Verás la página de inicio de la plantilla MVC. ¡Felicitaciones, tu primera app MVC está corriendo!

(1.5) Controladores y Acciones: El Corazón de la Interacción

- **Controladores (Controller):**

- Son clases C# que **heredan** de la clase base **Microsoft.AspNetCore.Mvc.Controller**. Esta clase base les proporciona acceso a funcionalidades útiles (como el método **View()**, **Redirect()**, acceso a datos de la petición, etc.).
- **Convención de Nombres:** Deben terminar con el sufijo **Controller** (ej: **HomeController**, **ProductosController**). El sistema de routing usa la parte anterior a "Controller" para encontrarlo (**Home**, **Productos**).
- **Propósito:** Agrupar lógicamente un conjunto de acciones relacionadas con una entidad o funcionalidad (ej: **ProductosController** manejaría acciones como listar productos, ver detalle de un producto, añadir producto, etc.).

- **Métodos de Acción (Action Methods):**

- Son métodos **públicos** dentro de una clase Controlador.
- Cada acción representa una operación específica que puede ser invocada por una URL a través del routing.

- **Tipo de Retorno:** Generalmente devuelven un tipo que implementa la interfaz `ActionResult`. Esto le da flexibilidad a la acción para decidir qué tipo de respuesta generar (mostrar una vista, redirigir a otra URL, devolver datos JSON, devolver un error, etc.).
- **ViewResult y el Método View():** El tipo de resultado más común para devolver HTML es `ViewResult`. La forma más fácil de crear un `ViewResult` es llamando al método `View()` heredado de la clase `Controller`.
 - `return View();`: Le dice a MVC: "Busca un archivo de vista `.cshtml` que coincida por convención con el nombre de este controlador y esta acción, y renderízalo usando el layout por defecto".
 - *Convención de Búsqueda de Vistas:* Si estás en la acción `Index` del `HomeController`, `return View();` buscará primero en `Views/Home/Index.cshtml`. Si no lo encuentra, buscará en `Views/Shared/Index.cshtml`.

- **Ejemplo - Creando un Controlador Básico:**

1. En Visual Studio, haz clic derecho en la carpeta `Controllers` en el Explorador de Soluciones.
2. Selecciona `Agregar > Controlador...`
3. Elige "Controlador de MVC - Vacío". Haz clic en "Agregar".
4. Dale un nombre que termine en `Controller`, por ejemplo: `SaludoController`. VS creará el archivo `SaludoController.cs`.
5. Modifica el código para que se vea así:

```
using Microsoft.AspNetCore.Mvc; // Necesario para Controller y
ActionResult

namespace MiPrimeraAppMVC.Controllers // Asegúrate que el namespace sea
correcto
{
    public class SaludoController : Controller // Hereda de Controller
    {
        // Acción por defecto si no se especifica en la URL (por
        routing)
        public IActionResult Index()
        {
            // Simplemente le dice a MVC que busque y renderice la
            vista
            // llamada "Index" dentro de la carpeta "Views/Saludo"
            // (o "Views/Shared" si no la encuentra allí).
            return View();
        }

        // Otra acción de ejemplo
        public IActionResult Despedida()
        {
            return View(); // Buscará Views/Saludo/Despedida.cshtml
        }
    }
}
```

```
}
}
```

(Nota: Aún no hemos creado las vistas, así que si intentas navegar a `/Saludo` o `/Saludo/Despedida` ahora, obtendrás un error indicando que no se encontró la vista. Eso lo haremos en la siguiente parte).

Parte 2: Routing, Vistas con Razor y Pasaje de Datos Básico

(2.1) Enrutamiento (Routing) Convencional en Detalle

El routing es el mecanismo que conecta las URLs que escribe el usuario (o a las que apuntan los enlaces) con el código específico (Controlador y Acción) que debe manejar esa petición.

- **Configuración (Recordatorio):** Se define en `Program.cs` usando `app.MapControllerRoute`.

```
app.MapControllerRoute(
    name: "default", // Un nombre descriptivo para la ruta
    pattern: "{controller=Home}/{action=Index}/{id?}" // La plantilla
);
```

- **La Plantilla de Ruta (`pattern`):** Define cómo se interpretan los segmentos de la URL.
 - `{controller=Home}`: Busca un segmento en la URL. Si lo encuentra (ej: `/Productos/...`), usará "Productos" como nombre del controlador (buscando `ProductosController`). Si **no** lo encuentra (ej: la URL es solo `/`), usará el valor por defecto "Home" (buscando `HomeController`).
 - `{action=Index}`: Busca el siguiente segmento. Si lo encuentra (ej: `/Home/Privacy`), usará "Privacy" como nombre de la acción (buscando el método `Privacy()` en `HomeController`). Si **no** lo encuentra (ej: la URL es `/Home` o `/`), usará el valor por defecto "Index" (buscando el método `Index()`).
 - `{id?}`: Busca un tercer segmento. Si lo encuentra (ej: `/Producto/Detalle/5`), intentará pasar ese valor ("5") como argumento a un parámetro llamado `id` en el método de acción (ej: `public IActionResult Detalle(int id)`). El signo `?` lo hace **opcional**: si no está presente en la URL, no pasa nada, el parámetro `id` tomará su valor por defecto (o será nulo si es un tipo que permite nulos).
- **Ejemplos de Mapeo con la Ruta por Defecto:**
 - URL: `/` -> Controlador: `Home`, Acción: `Index`
 - URL: `/Home` -> Controlador: `Home`, Acción: `Index`
 - URL: `/Home/Index` -> Controlador: `Home`, Acción: `Index`
 - URL: `/Home/Privacy` -> Controlador: `Home`, Acción: `Privacy`
 - URL: `/Saludo` -> Controlador: `Saludo`, Acción: `Index`
 - URL: `/Saludo/Despedida` -> Controlador: `Saludo`, Acción: `Despedida`
 - URL: `/Producto/Ver/10` -> Controlador: `Producto`, Acción: `Ver`, Parámetro `id` = 10

(2.2) Vistas y el Motor Razor

Las Vistas son las responsables de generar el HTML que se enviará al navegador. En ASP.NET Core MVC, se escriben usando la sintaxis **Razor** en archivos con extensión `.cshtml`.

- **¿Qué es Razor?** Es un motor de plantillas (template engine) que permite **mezclar código HTML con código C#** de forma fluida y legible dentro del mismo archivo `.cshtml`. El servidor procesa este archivo, ejecuta el código C# y lo intercala con el HTML para producir el HTML final puro que se envía al cliente.
- **Ubicación por Convención:** Es crucial guardar los archivos `.cshtml` en el lugar correcto para que MVC los encuentre automáticamente cuando una acción retorna `View()`.
 - `Views/[NombreControlador]/[NombreAccion].cshtml`: Para vistas específicas de una acción.
 - `Views/Shared/`: Para vistas parciales, layouts u otras vistas que puedan ser reutilizadas por varios controladores.
- **Sintaxis Razor Básica:** El carácter `@` marca la transición de HTML a C#.

```
<p>Hola, @ViewBag.NombreUsuario!</p>
<p>El año actual es: @DateTime.Now.Year</p>
<p>Resultado de 5 + 3: @(5 + 3)</p> <!-- Paréntesis para expresiones
más complejas -->
```

- **Bloques de Código:** Para escribir múltiples líneas de código C#. Las variables declaradas dentro de un bloque `@ { ... }` son locales a ese bloque de la vista.

```
@{
    string mensaje = "Este mensaje se define en C#.";
    int contador = 0;
    // Se pueden usar estructuras de control C#
    for (int i = 0; i < 3; i++) {
        contador++;
    }
}
<!-- Ahora podemos usar las variables en el HTML -->
<p>@mensaje</p>
<p>El contador llegó a: @contador</p>
```

- **Mezclando HTML y C# en Bucles/Condicionales:** Razor es bueno detectando HTML dentro de bloques C#.

```
@{
    List<string> nombres = new List<string> { "Ana", "Luis", "Eva" };
}
<ul>
    @foreach (string nombre in nombres)
    {
        <li>¡Hola, @nombre!</li> <!-- HTML dentro del bucle -->
    }
}
```



```

    }
</ul>

@if (DateTime.Now.Hour < 12)
{
    <p>¡Buenos días!</p>
} else {
    <p>Buenas tardes/noches.</p>
}

```

- **Comentarios Razor:** `@*` Este comentario no se envía al navegador `*@` (a diferencia de los comentarios HTML `` que sí viajan).
- **Codificación HTML Automática (¡Seguridad!):** Por defecto, Razor **codifica en HTML** cualquier salida de expresiones C#. Si una variable contiene `<script>alert('hack!');</script>`, al hacer `@variable` en la vista, se mostrará literalmente el texto `<script>alert('hack!');</script>` en el HTML final, **no** se ejecutará el script. Esto es una protección fundamental contra ataques **Cross-Site Scripting (XSS)**. Si *realmente* necesitas mostrar HTML que tienes en una variable sin codificar (¡sé muy cuidadoso con la fuente de ese HTML!), puedes usar `@Html.Raw(miVariableHtml)`.

• Creando una Vista:

1. Asegúrate de tener el `SaludoController` con la acción `Index()` que retorna `View()`.
2. En el Explorador de Soluciones, haz clic derecho en la carpeta `Views`.
3. Selecciona `Agregar > Nueva Carpeta`. Nómbrala exactamente igual que tu controlador pero sin el sufijo "Controller" (en este caso, `Saludo`).
4. Haz clic derecho en la nueva carpeta `Views/Saludo`.
5. Selecciona `Agregar > Vista...`
6. Elige "Vista Razor - Vacía". Haz clic en "Agregar".
7. Nombra el archivo exactamente igual que tu método de acción: `Index.cshtml`. Visual Studio creará el archivo.
8. Escribe contenido HTML y algo de Razor básico dentro:

```

@{
    // Podemos definir el título que usará el Layout
    ViewData["Title"] = "Página de Saludo";
    // ViewBag.Title = "Página de Saludo"; // También funciona si el
    Layout usa ViewBag
}

<h1>¡Bienvenido a la Página de Saludo!</h1>
<p>Esta es la vista generada por la acción Index del SaludoController.
</p>
<p>La hora actual del servidor es: @DateTime.Now.ToString("HH:mm:ss")
</p>

```

9. Ejecuta la aplicación (F5) y navega a la URL `/Saludo` (o `/Saludo/Index`). Deberías ver tu vista renderizada.

(2.3) Layouts y Vistas Compartidas: Evitando Repetición

Sería muy ineficiente repetir toda la estructura `<html>`, `<head>`, `<body>`, el menú de navegación, el pie de página, etc., en cada uno de los archivos `.cshtml` de tu aplicación. Para eso existen los **Layouts**.

- **_Layout.cshtml (La Plantilla Maestra):**
 - Por convención, se encuentra en `Views/Shared/`. El guion bajo `_` indica que no es una vista directamente accesible.
 - Contiene toda la estructura HTML común que quieres que tengan todas (o la mayoría) de tus páginas.
 - Incluye el `<!DOCTYPE>`, `<html>`, `<head>` (con enlaces a CSS/JS comunes, metas, etc.) y `<body>` (con elementos como `<header>`, `<nav>`, `<footer>`).
- **@RenderBody():**
 - Es una directiva Razor **obligatoria** que se coloca **una sola vez** dentro del `<body>` del archivo `_Layout.cshtml`.
 - Actúa como un **marcador de posición**. Cuando MVC renderiza una vista específica (como `Index.cshtml`), toma el contenido de esa vista y lo **inyecta** en el lugar donde está `@RenderBody()` dentro del Layout.
- **_ViewStart.cshtml (Configuración por Defecto):**
 - Por convención, se encuentra en la raíz de la carpeta `Views/`. El guion bajo `_` también indica que no es una vista directa.
 - El código C# dentro de este archivo se ejecuta **automáticamente antes** que el código de cualquier vista `.cshtml` ubicada en la misma carpeta o en subcarpetas.
 - Su uso más común es establecer la **página de Layout por defecto** para todas las vistas bajo `Views/`:

```
@{
    Layout = "_Layout"; // Le dice a todas las vistas que usen
    Views/Shared/_Layout.cshtml
}
```

- Una vista individual puede **sobrescribir** este layout por defecto si lo necesita, simplemente añadiendo `@{ Layout = "_OtroLayout"; }` o `@{ Layout = null; }` (para no usar ningún layout) al principio de su propio archivo `.cshtml`.

(2.4) Pasando Datos del Controlador a la Vista: ViewBag (Introducción)

Necesitamos enviar datos desde la lógica del Controlador hacia la Vista para que esta los muestre. **ViewBag** es una de las formas de hacerlo, especialmente para datos pequeños y simples.

- **¿Qué es ViewBag?** Es una propiedad de tipo `dynamic` de la clase base `Controller`. Ser "dinámico" significa que puedes agregarle propiedades en tiempo de ejecución sin declararlas previamente. Funciona como un contenedor temporal para pasar datos a la vista asociada.
- **¿Cómo se usa?**
 1. **En el Método de Acción (Controlador):** Simplemente asigna un valor a una propiedad "inventada" en `ViewBag`. El nombre de la propiedad lo eliges tú.

```
public IActionResult Index()
{
    // Creamos propiedades dinámicas en ViewBag
    ViewBag.Mensaje = "¡Saludos desde el Controlador!";
    ViewBag.FechaActual = DateTime.Today;
    ViewBag.CantidadAlumnos = 25;

    List<string> materias = new List<string> { "Programacion .NET",
    "Bases de Datos", "Laboratorio IV" };
    ViewBag.ListaMaterias = materias; // Podemos pasar colecciones
    también

    return View(); // La vista 'Index.cshtml' tendrá acceso a estos
    datos
}
```

2. En la Vista Razor (.cshtml): Accede a los valores usando la misma sintaxis

@ViewBag.NombrePropiedad.

```
@{
    ViewData["Title"] = "Página de Saludo";
}

<h1>Saludo</h1>

<p>@ViewBag.Mensaje</p>
<p>Hoy es: @ViewBag.FechaActual.ToShortDateString()</p>
<p>Somos @ViewBag.CantidadAlumnos alumnos.</p>

<h3>Materias:</h3>
<ul>
    @foreach (string materia in ViewBag.ListaMaterias)
    {
        <li>@materia</li>
    }
</ul>

<!-- Ojo si la propiedad no existe: --%>
<!-- <p>@ViewBag.DatoInexistente</p> --%> <!-- Esto daría un error en
tiempo de ejecución --%>
```

- **Ventajas y Desventajas (¡Importante Repaso!)**

- **Ventaja:** Muy simple y rápido para pasar datos ocasionales o auxiliares (ej: el título de la página, un mensaje de estado temporal).
- **Desventajas Fundamentales:**
 - **Sin Seguridad de Tipos (Not Statically Typed):** Es *dynamic*. El compilador no verifica si la propiedad que intentas leer en la vista (@ViewBag.Mensaje) realmente fue asignada en el controlador, ni de qué tipo es. Los errores (por escribir mal el nombre o por intentar

usar el dato como un tipo incorrecto) solo aparecen **en tiempo de ejecución**, cuando el usuario visita la página.

- **Sin IntelliSense:** El editor de código (Visual Studio) no puede ayudarte a autocompletar los nombres de las propiedades de `ViewBag` ni a detectar errores de tipeo mientras escribes.
- **Requiere Casting:** Si pasas objetos complejos, a menudo necesitarás hacer un *cast* (conversión explícita de tipo) en la vista para poder usar sus propiedades o métodos específicos, lo cual es engorroso y propenso a errores.
- **Propenso a Errores:** Es muy fácil cometer errores de tipeo en los nombres de las propiedades entre el controlador y la vista.
- **Alternativa Similar: ViewData**
 - Es otra propiedad disponible, pero funciona como un diccionario (`ViewDataDictionary`). Los datos se guardan y recuperan usando claves de tipo *string*.
 - Ej: `ViewData["Mensaje"] = "Hola";` en controlador, `@ViewData["Mensaje"]` en vista.
 - Sufre de las **mismas desventajas** que `ViewBag` (falta de seguridad de tipos, requiere casting, propenso a errores de tipeo en las claves).
- **El Camino Correcto (Próximos Pasos): ViewModels**
 - La forma **recomendada, robusta y profesional** de pasar datos desde el controlador a la vista en MVC es usando **Modelos de Vista (ViewModels)** fuertemente tipados.
 - ¿Qué es un *ViewModel*? Es una clase C# simple (POCO) que creas específicamente para contener *todos* los datos que una vista particular necesita mostrar. El controlador crea una instancia de este *ViewModel*, le carga los datos, y pasa *ese objeto completo* a la vista. La vista declara que espera recibir un objeto de ese tipo (`@model NombreViewModel`) y puede acceder a sus propiedades con seguridad de tipos e IntelliSense (`@Model.Propiedad`). ¡Mucho más seguro y mantenible! **Esto lo veremos en detalle en el próximo eje.**
 - **Conclusión sobre ViewBag/ViewData:** Útiles para casos muy puntuales y datos auxiliares. **NO los uses para pasar los datos principales o el modelo de tu vista.**

Conclusión

En esta introducción a ASP.NET Core MVC, hemos sentado las bases. Entendimos la arquitectura MVC y sus beneficios, cómo ASP.NET Core procesa las peticiones con su pipeline de middleware, y cómo crear y estructurar un proyecto MVC básico. Aprendimos sobre el enrutamiento convencional que conecta URLs con Controladores y Acciones. Vimos cómo las Acciones devuelven Vistas y cómo estas Vistas usan Razor para generar HTML dinámico, apoyándose en Layouts para la estructura común. Finalmente, exploramos una forma básica (aunque limitada) de pasar datos con ViewBag. Con estos fundamentos, estamos listos para profundizar en el Modelo, crear ViewModels robustos, interactuar con bases de datos y construir aplicaciones web MVC más completas.

Síntesis de una oración: Este material introduce la arquitectura MVC en ASP.NET Core, cubriendo la creación de proyectos, el flujo de peticiones mediante routing y middleware, la implementación de Controladores y Acciones, la generación de HTML dinámico con Vistas Razor y Layouts, y el paso básico de datos con ViewBag.

Síntesis de 5 puntos importantes:

1. **MVC** separa la aplicación en **Modelo** (datos/lógica), **Vista** (presentación HTML) y **Controlador** (manejo de peticiones/orquestación) para mejorar la organización y mantenibilidad.
2. **ASP.NET Core** procesa peticiones web a través de un **pipeline de middleware** configurable en *Program.cs*, donde también se registran servicios mediante **Inyección de Dependencias**.
3. El **Routing Convencional** mapea segmentos de URL (`{controller}/{action}/{id?}`) a clases `NombreController` y métodos de acción `public IActionResult NombreAccion()`.
4. Las **Vistas Razor** (`.cshtml`) mezclan HTML y C# (`@`) para generar la salida final, utilizando **Layouts** (`_Layout.cshtml`, `@RenderBody()`) para la estructura común.
5. **ViewBag** es un objeto dinámico para pasar datos simples del Controlador a la Vista, pero carece de seguridad de tipos y **no es recomendable** para datos complejos o principales (se prefieren `ViewModels`).

37784